

Task: Lift the red cube

```
1 # Code block 0
2 import numpy as np
3
4 # Step 1: Get a suitable grasp pose for the red cube.
5 grasp_pos, grasp_quat = sample_grasp_pose("red cube")
6
7 # Step 2: Open the gripper to prepare for grasping.
8 open_gripper()
9
10 # Step 3: Move to the grasp pose.
11 # We use a z_approach of 0.1 meters (10cm)
12 # to approach from above safely.
13 goto_pose(grasp_pos, grasp_quat, z_approach=0.1)
14
15 # Step 4: Close the gripper to grasp the object.
16 close_gripper()
17
18 # Step 5: Lift the object up.
19 # We will lift it by adding 0.2 meters
20 # to the current z-coordinate.
21 lift_pos = grasp_pos + np.array([0, 0, 0.2])
22 goto_pose(lift_pos, grasp_quat)
```

High-Level

```
1 # 1. Get current observation
2 obs = get_observation()
3 rgb = obs["robot0_robotview"]["images"]["rgb"]
4 depth = obs["robot0_robotview"]["images"]["depth"]
5 intrinsics = obs["robot0_robotview"]["intrinsics"]
6 camera_pose = obs["robot0_robotview"]["pose_mat"]
7
8 # 2. Segment the red cube
9 masks = segment_sam3_text_prompt(rgb, "red cube")
10 if not masks:
11     print("No red cube found.")
12     return
13
14 # Pick the mask with the highest score
15 best_mask_dict = max(masks, key=lambda x: x["score"])
16 mask = best_mask_dict["mask"]
17
18 # Ensure mask is integer for grasp planner
19 segmentation = mask.astype(np.int32)
20
21 # 3. Plan grasp
22 grasps, scores = plan_grasp(depth, K, seg)
23
24 # Select the best grasp based on score
25 best_idx = np.argmax(scores)
26 best_grasp_cam = grasps[best_idx]
27
28 # 4. Transform grasp to world frame
29 best_grasp_world = camera_pose @ best_grasp_cam
30
31 # 5. Extract position and orientation
32 target_pos = best_grasp_world[:3, 3]
33 target_rot = best_grasp_world[:3, :3]
34
35 # Convert rotation matrix to quaternion
36 rot = R.from_matrix(target_rot)
37 target_quat = rot.as_quat()
```

Perception

Object grounding

Grasp planning

Frame transform

Pose extraction

Low-Level